

TECHNICAL REPORT

Aluno(a) : Maria de Fátima Brandão de Andrade

Aluno(a) : Elyson Caique Alves Brito

1. Introdução

Neste trabalho foi desenvolvido um projeto completo de classificação utilizando o algoritmo K-Nearest Neighbors (KNN), aplicando conceitos de aprendizado de máquina supervisionado em um problema de detecção de websites phishing.

O phishing é um dos golpes mais comuns da internet atualmente. Nesse tipo de ataque, páginas falsas são criadas para enganar usuários e capturar informações pessoais, como senhas, dados bancários e informações confidenciais. Dessa forma, identificar automaticamente páginas maliciosas se tornou um problema importante na área de segurança da informação.

*O dataset utilizado foi o **Phishing Websites**, disponível na UCI Machine Learning Repository. Esse conjunto de dados possui atributos relacionados às características das URLs e ao comportamento dos websites, permitindo classificar se uma página é legítima ou phishing.*

Durante o desenvolvimento do projeto foram realizadas diversas etapas importantes, como:

- *exploração e análise inicial dos dados;*
- *tratamento de inconsistências;*
- *análise de atributos relevantes;*
- *normalização dos dados;*
- *treinamento do algoritmo KNN;*
- *análise da influência do parâmetro k ;*
- *validação cruzada;*
- *Grid Search;*
- *comparação entre diferentes técnicas de normalização.*

O principal objetivo do trabalho não foi apenas obter alta acurácia, mas compreender todo o processo envolvido na construção de um modelo de classificação.

2. Observações

Durante o desenvolvimento do projeto não foram encontrados valores ausentes no dataset, o que facilitou bastante a etapa de pré-processamento. Porém, foram identificadas muitas linhas duplicadas, totalizando 5.270 registros repetidos.

Inicialmente isso causou certa dúvida sobre manter ou remover esses dados. Após análise, optou-se pela remoção das duplicatas para evitar possíveis vieses no treinamento do modelo.

Outro detalhe importante foi a aplicação da normalização logarítmica. Como o dataset possuía valores negativos, foi necessário ajustar os dados antes de aplicar o logaritmo, somando uma constante em todos os atributos.

No geral, o desenvolvimento permitiu compreender na prática o impacto das etapas de pré-processamento e ajuste de hiperparâmetros no desempenho do modelo.

3. Resultados e discussão

3.1 Exploração Inicial do Dataset

A primeira etapa do projeto consistiu em explorar o dataset para compreender melhor suas características.

O conjunto de dados apresentava:

- *11.055 registros;*
- *30 atributos de entrada;*
- *uma variável alvo categórica;*
- *atributos representados numericamente.*

Também foi analisada a distribuição das classes:

- *Classe legítima (1): 6.157 registros;*
- *Classe phishing (-1): 4.898 registros.*

Apesar de existir uma pequena diferença entre as classes, o dataset apresentou um bom equilíbrio, o que ajuda no treinamento do modelo.

Além disso, verificou-se que todos os atributos estavam organizados em formato numérico, característica importante para o funcionamento do algoritmo KNN.

3.2 Tratamento de Valores Ausentes e Inconsistências

Foram realizadas verificações para identificar:

- valores nulos;
- inconsistências;
- registros duplicados;
- valores inválidos.

Não foram encontrados valores ausentes ou inconsistências aparentes. Entretanto, foi identificada uma quantidade muito alta de registros duplicados.

Após remover as duplicatas, o dataset passou de 11.055 para 5.849 registros.

Essa etapa foi importante porque registros repetidos podem influenciar artificialmente o desempenho do modelo, especialmente em algoritmos baseados em distância, como o KNN.

3.3 Análise de Correlação

Com o objetivo de entender quais atributos eram mais relevantes para o problema, foi calculada a correlação entre as variáveis e a classe alvo.

Os atributos com maior correlação foram:

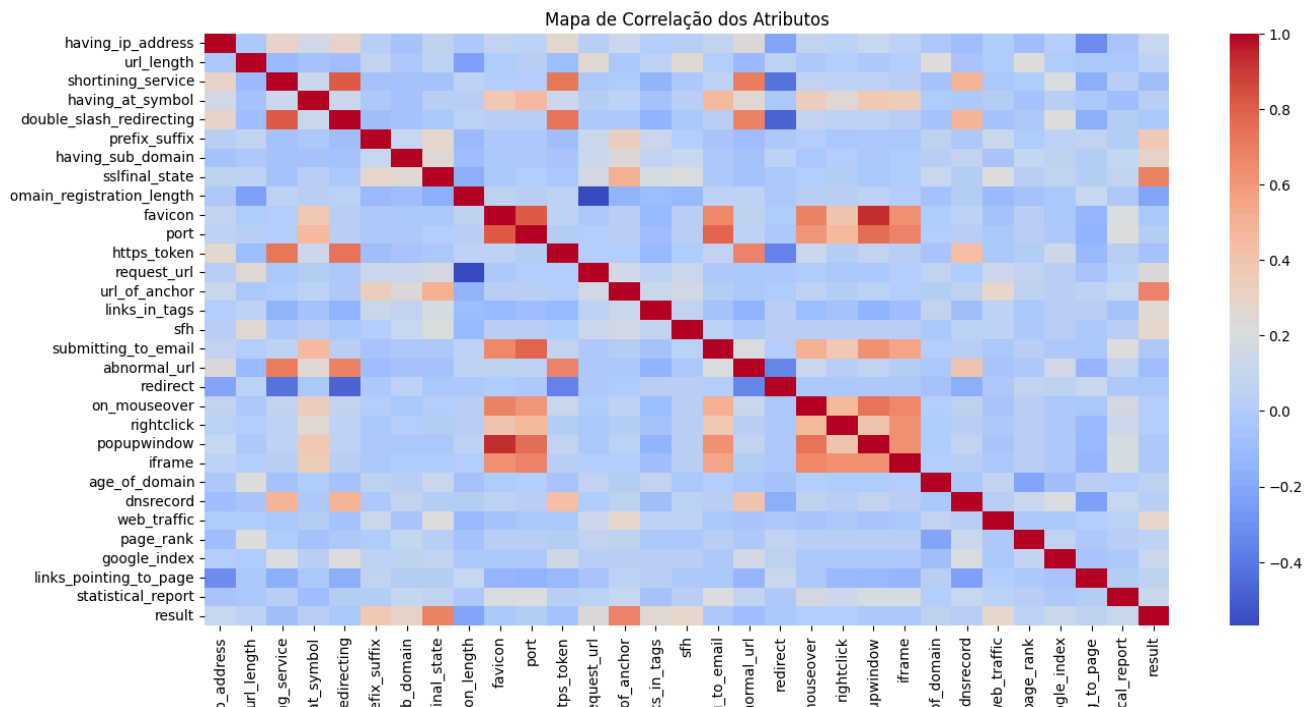
Atributo	Correlação
sslfinal_state	0.693
url_of_anchor	0.679
prefix_suffix	0.381
having_sub_domain	0.296
web_traffic	0.281

Tabela 1: Atributos mais correlacionados com a classe

Os resultados mostraram que características relacionadas à segurança do domínio e ao comportamento da URL possuem grande influência na classificação.

3.3.1 Heatmap de Correlação

A figura abaixo apresenta o mapa de correlação dos atributos.



Analisando o heatmap, percebe-se que alguns atributos possuem correlação positiva significativa com a variável alvo. Isso indica que esses atributos ajudam bastante o modelo a distinguir páginas legítimas de páginas phishing.

3.4 Separação dos Dados

Após o pré-processamento, os dados foram separados em:

- atributos de entrada (X);
- variável alvo (y).

As dimensões finais ficaram:

- X : (5849, 30);
- y : (5849,).

Posteriormente, os dados foram divididos em conjuntos de treino e teste:

- 80% para treino;
- 20% para teste.

3.5 Normalização dos Dados

Como o algoritmo KNN utiliza cálculo de distância entre pontos, a normalização dos dados é extremamente importante.

Sem normalização, atributos com valores maiores podem influenciar excessivamente os cálculos de distância, prejudicando o desempenho do modelo.

Foram utilizadas três técnicas diferentes:

- *Min-Max;*
- *Z-Score;*
- *Transformação Logarítmica.*

Os resultados obtidos foram:

Normalização	Acurácia
Min-Max	0.9065
Z-Score	0.9104
Log	0.9101

Tabela 2: Comparação entre normalizações

A técnica Z-Score apresentou o melhor desempenho geral, embora a diferença entre os métodos tenha sido relativamente pequena.

3.6 Treinamento do Modelo KNN

O modelo foi inicialmente treinado utilizando:

- *$k = 5$;*
- *divisão treino/teste de 80/20.*

*A acurácia obtida foi de **92,31%**.*

Esse resultado mostrou que o modelo conseguiu aprender bem os padrões presentes no dataset.

3.7 Matriz de Confusão

A matriz de confusão obtida foi:

	Predito -1	Predito 1
Real -1	580	40
Real 1	50	500

Tabela 3: Matriz de confusão

Os resultados da matriz de confusão mostraram que o modelo conseguiu classificar corretamente a maioria dos exemplos.

Apesar de existirem alguns erros, o desempenho geral foi bastante satisfatório.

3.8 Classification Report

Os principais indicadores foram:

Classe	Precision	Recall	F1-score
-1	0.92	0.94	0.93
1	0.93	0.91	0.92

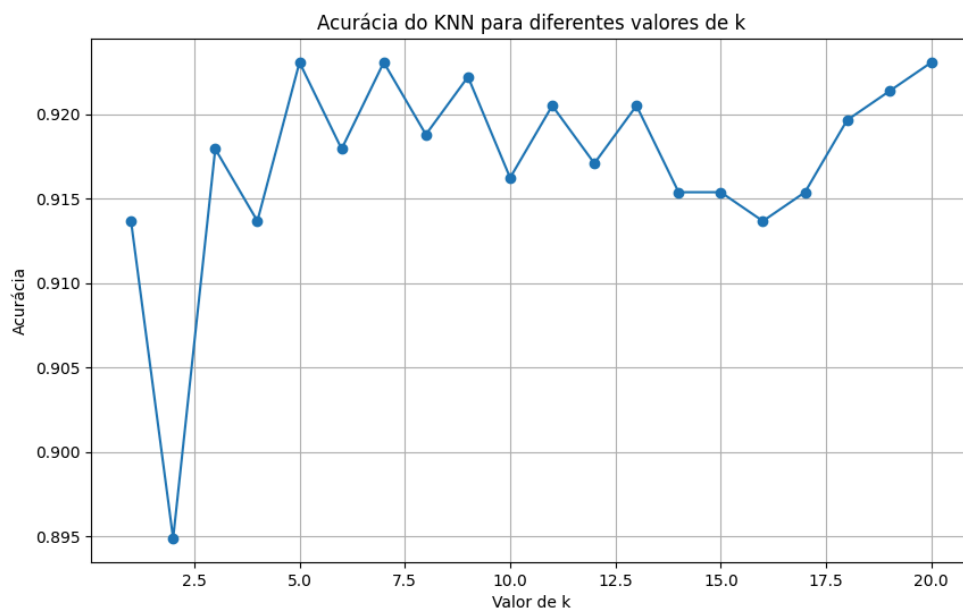
Tabela 4: Classification Report

Os valores obtidos indicaram que o modelo apresentou equilíbrio entre precisão e recall, conseguindo bons resultados em ambas as classes.

3.9 Influência do Parâmetro k

Foi realizada uma análise variando o valor de k entre 1 e 20.

O objetivo dessa etapa foi observar como diferentes quantidades de vizinhos afetam o desempenho do modelo.



Analisando os resultados, percebe-se que valores muito baixos de k apresentaram maior instabilidade.

Os melhores resultados ocorreram próximos de:

- $k = 5$;
- $k = 7$;
- $k = 20$.

Visualmente, o melhor resultado encontrado foi com $k = 5$.

3.10 Validação Cruzada

Para avaliar melhor a capacidade de generalização do modelo, foi aplicada validação cruzada com 5 folds.

O melhor resultado obtido foi:

- $k = 7$;
- *acurácia média* = 0.9077.

A validação cruzada apresentou resultados ligeiramente menores que os encontrados no conjunto de teste, o que era esperado, já que ela fornece uma avaliação mais robusta do modelo.

3.11 Grid Search

Também foi utilizado o Grid Search para encontrar automaticamente os melhores hiperparâmetros do KNN.

Os parâmetros testados foram:

- *número de vizinhos*;
- *métrica de distância*;
- *tipo de peso*.

Os melhores parâmetros encontrados foram:

- *metric* = manhattan;
- *n_neighbors* = 6;
- *weights* = distance.

*A melhor acurácia obtida foi de **0.9301**.*

Esse resultado foi superior ao modelo configurado manualmente, mostrando a importância da otimização automática de hiperparâmetros.

4. Conclusões

Os resultados obtidos neste trabalho foram bastante satisfatórios.

O algoritmo KNN apresentou excelente desempenho na classificação de websites phishing, alcançando acurácia superior a 92%.

Além disso, o projeto permitiu compreender melhor como etapas como pré-processamento, normalização e ajuste de hiperparâmetros influenciam diretamente os resultados finais.

A comparação entre diferentes valores de k mostrou que valores intermediários tendem a produzir modelos mais estáveis e equilibrados.



Também foi possível observar que técnicas como validação cruzada e Grid Search tornam a avaliação do modelo mais confiável e ajudam a encontrar configurações mais eficientes automaticamente.

Outro ponto importante foi perceber que pequenas mudanças no pré-processamento podem causar impacto relevante no desempenho do algoritmo.

De forma geral, os objetivos do projeto foram atingidos com sucesso e o trabalho contribuiu bastante para o entendimento prático do funcionamento de modelos de classificação supervisionada.

5. Próximos passos

Como continuidade do projeto, algumas melhorias podem ser implementadas:

- *testar outros algoritmos de classificação, como Random Forest e SVM;*
- *utilizar técnicas automáticas de seleção de atributos;*
- *avaliar o modelo em datasets maiores;*
- *aplicar técnicas de balanceamento de classes;*
- *desenvolver uma aplicação em tempo real para detecção de phishing.*

Essas melhorias podem aumentar ainda mais a capacidade de generalização e robustez do modelo.